

SwimFest India — Project Report

Golden Non-Medalist Championship 2026

1. Project Overview

Project Name: SwimFest India — Swimming Competition Management System

Client: SwimFest India

Developer: Vishnu Reddy

Technology Stack: Flask (Python), Supabase (PostgreSQL), HTML/CSS/JavaScript

Live URL: <https://swimfest-india.onrender.com>

GitHub: <https://github.com/VISHNUREDDY352/swimfest-india>

Purpose

SwimFest India is a full-stack web application for managing swimming competitions. It handles swimmer registration, event booking, payment processing, heat sheet generation, and result management for the Golden Non-Medalist Championship 2026 — a competition exclusively for swimmers who have never won an individual medal at State, Zonal, National, or major School Games.

2. Features

For Swimmers

- Account creation with full profile (personal, school, academy, parent details)
- Age group auto-detection from date of birth
- Tournament registration wizard (7-step booking flow)
- Event selection (max 3 individual + 1 relay)
- Seed time entry
- Online payment via Razorpay (UPI, card, net banking)
- Email + WhatsApp confirmation after booking
- Player profile page with booking history and event entries
- Participation certificate (available after event completion)
- Eligibility checker on home page

For Administrators

- Secure admin login (separate credentials)
- Dashboard with live stats (swimmers, bookings, entries, academies)
- Create, edit, and manage tournaments
- Update tournament status (Open / Upcoming / Ongoing / Closed / Completed)

- View all registered swimmers and edit profiles
- View all bookings and event entries
- Generate heat sheets (lane assignment based on seed time)
- Upload results (final time and placement)
- Export heat sheet as CSV or PDF

General

- Live ticker banner with event details
- Announcements section (dynamic from DB)
- Swimmer directory (loaded from DB)
- Registration deadline countdown timer
- Event cards sorted by soonest deadline
- Share event card via WhatsApp
- Venue popup with Google Maps directions
- Swimming tips and How It Works section
- Past event results
- FAQ section
- Contact section
- Fully responsive (desktop + mobile)

3. Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, Vanilla JavaScript
Backend	Python 3, Flask 3.0.3
Database	Supabase (PostgreSQL)
Hosting	Render (free tier)
Payment	Razorpay (test mode)
Email	Gmail SMTP
PDF	html2canvas + jsPDF (certificates)
Version Control	Git + GitHub

4. System Architecture

```
graph TD
    Browser["Browser (HTML/CSS/JS)"] -- "HTTP requests" --> Flask["Flask Backend (app.py)"]
    Flask -- "Supabase Python Client" --> Supabase["Supabase"]
```

v
Supabase PostgreSQL (Cloud DB)

- Flask serves all frontend static files from the root directory
- All API routes are prefixed with `/api/`
- Root URL `/` redirects to `login.html`
- Authentication is session-based using localStorage (swimmer_id)

5. Database Schema

Table: swimmers

Column	Type	Description
swimmer_id	TEXT PK	e.g. SWM-2026-XXXX
full_name	TEXT	Swimmer's official name
gender	TEXT	Boys / Girls
dob	TEXT	Date of birth (YYYY-MM-DD)
year_of_birth	INTEGER	Derived from DOB
age_group	TEXT	U-10 / U-12 / U-14 / U-16
mobile	TEXT	Swimmer's mobile
parent_name	TEXT	Parent/Guardian name
parent_relationship	TEXT	Father / Mother / Guardian
parent_mobile	TEXT	Parent contact
parent_email	TEXT	Parent email
academy_id	TEXT FK	References academies
school_name	TEXT	School name
school_board	TEXT	CBSE / ICSE / State Board
school_class	TEXT	Grade/Class
address	TEXT	Full address
nationality	TEXT	Default: Indian
eligibility	TEXT	Non-Medalist
password_hash	TEXT	SHA-256 hashed password
status	TEXT	Active
created_at	TIMESTAMPTZ	Auto

Table: academies

Column	Type	Description
academy_id	TEXT PK	e.g. ACD-001
name	TEXT	Academy name
short_name	TEXT	Abbreviation
head_coach	TEXT	Coach name
mobile	TEXT	Contact
email	TEXT	Email
city	TEXT	City
address	TEXT	Address
status	TEXT	Active

Table: tournaments

Column	Type	Description
tournament_id	TEXT PK	e.g. GNMC-2026
name	TEXT	Tournament name
venue	TEXT	Venue address
start_date	TEXT	YYYY-MM-DD
end_date	TEXT	YYYY-MM-DD
reg_deadline	TEXT	Registration deadline
fee_per_event	REAL	Per event fee (INR)
relay_fee	REAL	Relay fee
gst_rate	REAL	0.18 = 18%
lanes	INTEGER	Pool lanes (6/8/10)
status	TEXT	Open/Upcoming/Ongoing/Closed/Completed

Table: bookings

Column	Type	Description
booking_id	TEXT PK	e.g. BKG-XXXXXXXX
swimmer_id	TEXT FK	References swimmers
tournament_id	TEXT FK	References tournaments
total_fee	REAL	Subtotal
gst	REAL	GST amount
total_paid	REAL	Total paid
payment_method	TEXT	upi/card/netbanking
payment_status	TEXT	Pending / Paid
relay_opted	INTEGER	0 or 1
consent_*	INTEGER	Consent flags

Table: event_entries

Column	Type	Description
entry_id	TEXT PK	e.g. ENT-XXXXXXXX
booking_id	TEXT FK	References bookings
swimmer_id	TEXT FK	References swimmers
tournament_id	TEXT FK	References tournaments
event_name	TEXT	e.g. 100m Freestyle
event_code	TEXT	FS/BK/BR/BF/IM/RLY
distance	TEXT	50m / 100m / 200m
seed_time	TEXT	MM:SS.ss or NT
heat_no	INTEGER	Assigned heat
lane_no	INTEGER	Assigned lane
final_time	TEXT	Race result
overall_place	INTEGER	Final placement
status	TEXT	Registered / OK / DQ

Table: heat_sheets

Column	Type	Description
hs_id	TEXT PK	Generated ID
tournament_id	TEXT FK	References tournaments
event_name	TEXT	Event name
category	TEXT	Age group
gender	TEXT	Boys/Girls
heat_no	INTEGER	Heat number
lane_no	INTEGER	Lane number
swimmer_id	TEXT	Swimmer reference
swimmer_name	TEXT	Denormalized name
academy_name	TEXT	Denormalized academy
seed_time	TEXT	Seed time
start_time	TEXT	Scheduled start

6. API Endpoints

Method	Endpoint	Description
GET	/api/home	Combined home page data (cached)
POST	/api/auth/login	Login (swimmer + admin)
GET	/api/academies	List active academies
POST	/api/academies	Create academy
GET	/api/swimmers	List all swimmers
GET	/api/swimmers/:id	Get swimmer by ID
PUT	/api/swimmers/:id	Update swimmer profile
POST	/api/swimmers	Register new swimmer
GET	/api/tournaments	List tournaments
POST	/api/tournaments	Create tournament
PUT	/api/tournaments/:id	Edit tournament
DELETE	/api/tournaments/:id	Delete tournament
PUT	/api/tournaments/:id/status	Update status
GET	/api/bookings	List all bookings
POST	/api/bookings	Create booking
GET	/api/bookings/:id	Get booking details
POST	/api/payment/create-order	Create Razorpay order
POST	/api/payment/verify	Verify payment signature
GET	/api/payment/config	Get Razorpay key
POST	/api/heatsheet/generate/:tid	Generate heat sheet
GET	/api/heatsheet/:tid	Get heat sheet
GET	/api/results/:tid	Get results
POST	/api/results/update	Update result entry
GET	/api/stats	Dashboard statistics

7. Age Categories

Category	Birth Years	Individual Events	Relay
U-10	2016–2017	50m FS, 100m FS, 50m BK, 50m BR, 50m BF, 100m IM	4×50m
U-12	2014–2015	50m FS, 100m FS, 50m BK, 50m BR, 50m BF, 100m IM	4×50m
U-14	2012–2013	50m FS, 100m FS, 100m BK, 100m BR, 100m BF, 200m IM	4×100m
U-16	2010–2011	50m FS, 100m FS, 100m BK, 100m BR, 100m BF, 200m IM	4×100m

Age cut-off: 31 December 2026

Max events per swimmer: 3 individual + 1 relay

8. Payment Flow

1. User selects events → wizard calculates total (events × ₹300 + GST 18%)
2. Frontend calls `/api/payment/create-order` with amount
3. Backend creates Razorpay order and returns `order_id`
4. Razorpay checkout opens in browser
5. User pays via UPI/card/net banking
6. On success, Razorpay returns `payment_id` and `signature`
7. Frontend calls `/api/payment/verify` to validate signature
8. Backend marks booking as Paid and sends confirmation email

Razorpay: Test mode (Key: rzp_test_T3l0Fu0bS5Yvi6)

9. Heat Sheet Generation

1. Admin clicks "Generate Heat Sheet" for a tournament
2. Backend fetches all event_entries for the tournament
3. Entries are grouped by `age_group + gender + event_name`
4. Within each group, swimmers are sorted by seed time (NT last)
5. Groups are split into heats of `lanes` size
6. Lane assignment uses centre-out seeding pattern:
7. 8 lanes: fastest → lanes 4,5,3,6,2,7,1,8
8. Start times are calculated at 3 min/heat + 5 min break between events
9. Heat sheet records saved to `heat_sheets` table

10. Security

- Admin credentials: stored in backend code (not in DB)
- Swimmer passwords: SHA-256 hashed before storage
- Razorpay payment signature verified server-side (HMAC-SHA256)
- CORS enabled for API routes

- Row Level Security (RLS) enabled on all Supabase tables
- Service role key (bypasses RLS) used only in backend

11. Deployment

Local Development

```
cd c:\Users\vishn\DEV
python backend/app.py
# Access at http://localhost:5000
```

Production (Render)

- **Platform:** Render.com (free tier)
- **Start Command:** `gunicorn --chdir backend app:app`
- **Build Command:** `pip install -r requirements.txt`
- **Environment Variable:** `PYTHON_VERSION=3.11.0`
- **Auto Deploy:** On every push to `main` branch

Database

- **Platform:** Supabase (free tier, PostgreSQL)
- **Project ID:** gacgkcsjddqikjudqhto
- **Region:** US East

12. File Structure

```
DEV/
├── backend/
│   ├── app.py           # Flask backend + all API routes
│   ├── email_service.py # Gmail SMTP email sender
│   ├── supabase_config.py # Supabase credentials
│   └── setup_supabase.py # DB seed script
├── index.html           # Home page
├── login.html           # Login / Register
├── signup.html          # Create account
├── player.html          # Swimmer profile
├── admin.html           # Admin panel
├── heatsheet.html       # Heat sheet generator
├── certificate.html      # Participation certificate
├── home.css             # Main styles
├── home.js              # Home page JS
├── app.js               # Registration wizard JS
├── heatsheet.js         # Heat sheet JS
├── player.css           # Profile page styles
├── heatsheet.css        # Heat sheet styles
├── requirements.txt      # Python dependencies
├── Procfile             # Render start command
├── docs/
│   ├── Project_Report.md # This document
│   └── User_Manual.md    # User guide
```

Document prepared by: Vishnu Reddy

Project: SwimFest India 2026

Date: July 2026